

**** LAB ** Gestione di servizi e monitoraggio [31 marzo]**

Sito: [Virtuale](#)

Corso: Amministrazione di Sistemi T - Laboratorio di
Amministrazione di Sistemi T

Libro: ** LAB ** Gestione di servizi e monitoraggio [31 marzo]

Stampato da: Lorenzo Bernardini

Data: venerdì, 24 aprile 2026, 11:53

Sommario

1. at
2. watchdog
3. Esercizio: nice execution
4. Soluzione nice executon
5. manipolazione crontab
6. configurazione syslog
7. Esercizio: logconfig
8. Soluzione: logconfig
9. systemd units

1. at

Sulle VM at non è installato. Il pacchetto è **at**

Scheduling differito di un servizio, con possibilità di disinnescare

```
ATJOB=$(mktemp)
```

```
echo "/bin/comando" | at now + 30 minutes 2>&1 | grep '^job ' | cut -f2 -d' ' > $ATJOB
```

[se necessario de-schedulare prima dell'esecuzione:]

```
atrm $(cat $ATJOB)
```

2. watchdog

Se si vuole porre un limite al tempo di esecuzione di un task, senza ricorrere a un busy wait

```
ipotetico_comando_che_dura_a_lungo & PID=$!  
WD=$(mktemp)  
echo "/bin/kill $PID" | at now + $LIMITE minutes 2>&1 | grep '^job ' | cut -f1 -d' ' > $WD
```

[... spesso i watchdog sono disinnescati e reinnescati da determinati eventi ...]

Esempio: riavvia il sistema se non vede comparire messaggi in un file di log per 5 minuti consecutivi

```
WD=$(mktemp)  
function restartWatchdog() {  
    if [[ -s "$WD" && "$(atq | awk '{ print $1 }')" == "$(cat $WD)" ]] ; then  
        atrm $(cat $WD)  
    fi  
    echo "/usr/bin/shutdown -r now" | at now + 5 minutes 2>&1 | grep '^job ' | cut -f1 -d' ' > $WD  
}  
restartWatchdog  
tail -f /var/log/event.log | while read riga ; do  
    # delay restart for 5 more minutes  
    restartWatchdog  
    # do things  
done
```

3. Esercizio: nice execution

Se il carico del sistema è inferiore ad una soglia specificata come primo parametro dello script, lancia il comando specificato come secondo parametro.

Altrimenti, con at, rischedula il test dopo 2 minuti, e procede così finchè non riesce a lanciare il comando.

4. Soluzione nice executon

```
#!/bin/bash
# Se il carico del sistema è inferiore ad una soglia specificata
# come primo parametro dello script, lancia il comando specificato
# come secondo parametro.
# Altrimenti, con at, rischedula il test dopo 2 minuti, e procede
# così finché non riesce a lanciare il comando.

# configurazione del nome dello script
# come si può ricavare automaticamente senza ambiguità'
# dovute a path relativi che potrebbero confondere atd?
THIS=/home/vagrant/niceexec-base.sh
```

```
# testare se $1 è un numero
if ! [[ "$1" =~ ^[0-9]+$ ]] ; then
    echo "$1 non è un numero"
    exit 1
fi

# $2 deve essere eseguibile, file standard e con path assoluto
# per evitare problemi con l'environment di atd
if ! [[ -x "$2" && -f "$2" && "$2" =~ ^/ ]] ; then
    echo "$2 non è un eseguibile con path assoluto"
    exit 1
fi

# ipotesi semplificativa: solo la parte intera del carico a 1 minuto
# per farlo, devo sapere qual è il delimitatore dei decimali
# in accordo alla localizzazione attiva: man locale
LOAD=$(uptime | awk -F 'average: ' '{ print $2 }' | cut -f1 -d$(locale decimal_point))
if test $LOAD -lt "$1" ; then
    shift
    eval "$@"
else
    echo $THIS "$@" | at now +2 minutes
fi
```

5. manipolazione crontab

oltre a editare manualmente le tabelle, crontab permette anche di aggiornarle da script

esempio

```
PERIODO='*/10 8-18 * * *'  
COMANDO='/home/las/qualcosa.sh parametri vari"  
TMPTAB=$(mktemp)  
# dump della tabella e rimozione di eventuale riga che già contenga COMANDO, per evitare duplicati  
crontab -l | grep -vF "$COMANDO" > $TMPTAB  
echo "$PERIODO $COMANDO" >> $TMPTAB  
crontab $TMPTAB  
rm $TMPTAB
```

6. configurazione syslog

```
# per aggiungere facility di log si può usare la directory /etc/rsyslog.d
# che contiene file di configurazione automaticamente inclusi
```

```
# es nuovo file:
echo "local4.=info /var/log/myevents.log" > /etc/rsyslog.d/mylog.conf
```

```
# es. disattivazione della facility aggiunta
mv /etc/rsyslog.d/mylog.conf /etc/rsyslog.d/mylog.off
```

```
# in ogni caso dopo qualsiasi modifica del genere mostrato sopra
systemctl restart rsyslog
```


7. Esercizio: logconfig

Si realizzi uno script che prende come parametri

- una stringa nel formato **facility.priority**
- un nome di file

e configuri automaticamente

- **syslog** per loggare i messaggi con l'etichetta specificata sul relativo file
- **cron** per mandare ogni 5 minuti l'output del comando **uptime** a syslog con l'etichetta specificata

8. Soluzione: logconfig

```
#!/bin/bash
```

```
# Si realizzi uno script che prende come parametri
#   una stringa nel formato facility.priority
#   un nome di file assoluto
# e configuri automaticamente
#   syslog per loggare i messaggi con l'etichetta specificata sul relativo file
#   cron per mandare ogni 5 minuti l'output del comando uptime
#   a syslog con l'etichetta specificata

# estensione proposta: come testare che i valori di facility e priority
# siano veramente ammissibili, non semplicemente stringhe?
[[ "$1" =~ ^[:alnum:]+\.[[:alnum:]]+$ ]] || {
    echo Il primo parametro deve essere una stringa nel formato facility.priority
    exit 1
}

[[ "$2" =~ ^/ ]] || {
    echo Il primo parametro deve essere un filename assoluto
    exit 2
}
```

```
T=$(mktemp)
```

```
echo -e "$1\t$2" > $T
sudo cp $T /etc/rsyslogd.d/logconfig.conf
sudo systemctl restart rsyslog.service
```

```
CRONCOMMAND="/usr/bin/uptime | /usr/bin/logger -p $1"
crontab -l | grep -Fv "$CRONCOMMAND" > $T
echo "*/5 * * * * $CRONCOMMAND" >> $T
crontab $T

rm -f $T
```

9. systemd units

Esempi reali da guardare per comprendere il ruolo delle direttive, controllare i log tramite journalctl

- unit di tipo oneshot, per lanciare alcuni comandi di configurazione con "start" e ripristinare lo stato precedente con "stop"

```
/lib/systemd/system/networking.service
```

- unit di tipo forking, per avviare un demone in background

```
/lib/systemd/system/dnsmasq.service
```

- unit con opzione Restart

```
/lib/systemd/system/atd.service  
/lib/systemd/system/cron.service  
/lib/systemd/system/rsyslog.service
```